🎓 Educational MCP Server Project: Understanding AI Tool Integration

📚 Educational Overview

This project is a comprehensive educational example that demonstrates how Large Language Models (LLMs) can interact with external systems through the **Model Context Protocol (MCP)**. It's designed to teach the fundamental concepts of AI tool integration, protocol design, and modern AI application architecture.

🧠 What You'll Learn

- **MCP Protocol Fundamentals** - How AI systems communicate with external tools
- **Stdio Transport Mechanism** - Inter-process communication between AI and tools
- **MCPHost Architecture** - Modern approach to AI tool management
- **Local AI Model Integration** - Using Ollama with Llama 3.2:1b
- **Real-world Tool Implementation** - Building practical AI tools with NestJS

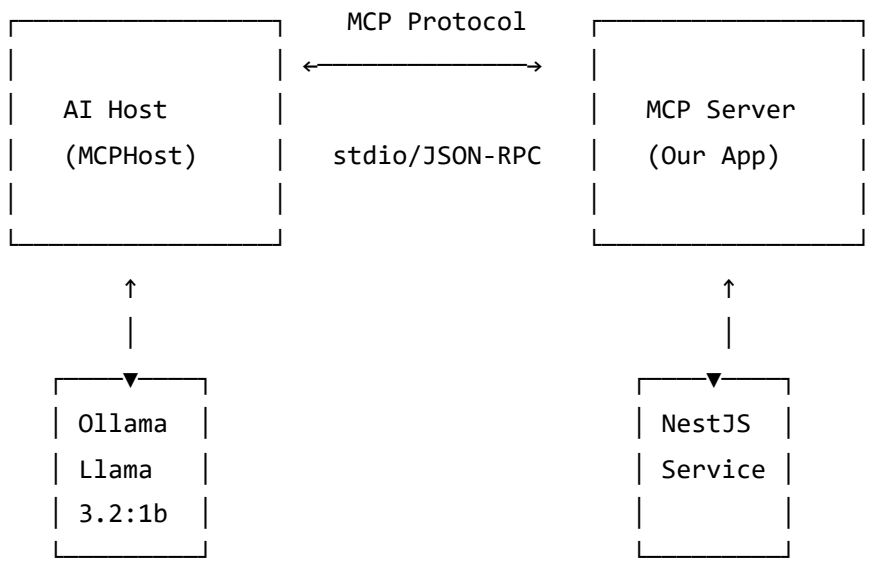
🔍 Understanding MCP (Model Context Protocol)

What is MCP?

The **Model Context Protocol (MCP)** is a standard that enables Large Language Models to:

- 🛠️ **Access external tools** and services safely
- 🔄 **Maintain context** across interactions
- 🔒 **Execute commands** with proper security boundaries
- 📡 **Communicate** with various systems and APIs

MCP Architecture Explained



Key MCP Components

1. **Host (MCPHost)**
   - Manages AI model (Ollama/Llama 3.2:1b)
   - Handles user conversations
   - Discovers and calls MCP tools
   - Formats responses back to users
2. **Server (Our NestJS App)**
   - Implements MCP protocol
   - Exposes tools via JSON-RPC
   - Handles actual system operations
   - Returns results to the host
3. **Transport (Stdio)**
   - Communication channel between host and server
   - Uses standard input/output streams
   - JSON-RPC message format
   - Bidirectional data flow

🚀 Why Use MCPHost with Ollama?

Traditional Approach Problems

```
# ❌ Complex Ollama-only setup
# Requires editing system config files:
# %APPDATA%/Ollama/config.json
{
  "experimental": {
    "mcp": {
      "servers": {
        "directory": {
          "command": ["node", "dist/main.js"],
          "args": [],
          "env": {}
        }
      }
    }
  }
}
```

MCPHost Solution Benefits

```
# ✅ Simple MCPHost configuration
model: "ollama:llama3.2:1b"
mcpServers:
  simple-directory-server:
    type: "local"
    command: ["node", "dist/main.js"]
```

Why MCPHost is Better:

- 🎯 **Simplified Configuration** - Single YAML file vs complex JSON configs
- 🔧 **Better Debugging** - Clear error messages and tool discovery
- 🔄 **Multiple Model Support** - Easy switching between AI providers
- 🛡️ **Reliable Protocol** - More stable than Ollama's experimental MCP
- 📊 **Rich Features** - Interactive commands, streaming, history
- 🌍 **Environment Variables** - Flexible configuration management

🔌 Understanding Stdio Transport

What is Stdio Transport?

Stdio (Standard Input/Output) transport is a communication method where:

- The **MCP server** reads from `stdin` and writes to `stdout`
- The **MCP host** writes to server's `stdin` and reads from `stdout`
- Messages are **JSON-RPC** formatted
- Communication is **bidirectional** and **asynchronous**

How Stdio Works in Our Project

```
// In our main.ts - MCP Server Side
const transport = new StdioServerTransport();
await server.connect(transport);
// Server listens on stdin, responds on stdout


# In .mcphost.yml - MCPHost Side
mcpServers:
  simple-directory-server:
    type: "local"
    command: ["node", "dist/main.js"]  # Starts our server
    # MCPHost connects via stdin/stdout
```

Message Flow Example

1. **User asks**: "List files in C:/Users"
2. **MCPHost** determines it needs `list_directory` tool
3. **MCPHost** → **Server** (via stdin):

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "list_directory",
    "arguments": {"path": "C:/Users"}
  }
}
```

4. **Server processes** request in `DirectoryService`
5. **Server → MCPHost** (via stdout):

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [{"type": "text", "text": "drwxr-... user1\ndrwxr-... user2"}]
  }
}
```

6. **MCPHost** formats response for user

🏗️ Project Architecture Deep Dive

Core Components Explained

1. **DirectoryService** (`src/directory/directory.service.ts`)

Purpose: The business logic layer that handles directory operations

Key Features:

- 🛡️ **Path Sanitization** - Handles Windows path corruption from MCP
- 🔧 **Git Bash Integration** - Uses Unix-style `ls` command on Windows
- 🔄 **Fallback Mechanism** - PowerShell backup if Git Bash fails
- ⚡ **Error Handling** - Comprehensive error management

Why This Design?:

```
// Path corruption handling - real-world problem
// Input: "C:\\\\Users\\\\lemys\\\\.\\\\lopez"
// Output: "C:/Users/lemys.lopez"
cleanPath = cleanPath.replace(/\\\\/g, '\\');     // Fix escaping
cleanPath = cleanPath.replace(/\\\.\\/g, '.');    // Fix .\ corruption
cleanPath = cleanPath.replace(/\\/g, '/');        // Unix paths
cleanPath = cleanPath.replace(/^\/([a-zA-Z]:)/, '$1'); // Fix drive paths
```

2. DirectoryController (`src/directory/directory.controller.ts`)

Purpose: HTTP REST interface (optional, for testing outside MCP)

```
@Controller('directory')
export class DirectoryController {
  @Get('list')
  async list(@Query('path') path?: string) {
    return this.directoryService.listDirectory(path || '.');
  }
}
```

Why Include This?:

- 🧪 **Testing** - Easy to test service without MCP complexity
- 🔍 **Debugging** - Direct HTTP access for troubleshooting
- 📚 **Learning** - Shows difference between HTTP and MCP interfaces
- 🔄 **Flexibility** - Same service, multiple interfaces

3. MCP Server (`src/main.ts`)

Purpose: MCP protocol implementation and stdio transport

Key MCP Handlers:

```
// Tool Discovery - MCPHost asks "what tools do you have?"
server.setRequestHandler(ListToolsRequestSchema, async () => {
  return {
    tools: [{
      name: 'list_directory',
      description: 'List contents of a directory using Git Bash ls command',
      inputSchema: { /* JSON Schema for validation */ }
    }]
  };
});

// Tool Execution - MCPHost says "run this tool with these params"
server.setRequestHandler(CallToolRequestSchema, async (request) => {
  const { name, arguments: args } = request.params;
  if (name === 'list_directory') {
    const result = await directoryService.listDirectory(args.path);
    return { content: [{ type: 'text', text: result }] };
  }
});
```

🎯 Why These Design Choices?

1. NestJS Framework

- 📦 **Dependency Injection** - Clean service organization
- 🛠️ **Modularity** - Easy to extend with more tools
- 🧪 **Testing** - Built-in testing framework
- 📚 **Learning** - Industry-standard enterprise patterns

2. Git Bash for Directory Listing

```
// Why not just Node.js fs.readdir()?
// Education: Shows how to integrate external tools
// Real-world: Many systems require calling external commands
// Cross-platform: Unix tools work consistently across platforms
const command = `"${gitBashPath}" -c "ls -la '${sanitizedPath}'"`;
```

3. Stdio Transport Over HTTP

- ⚡ **Performance** - No HTTP overhead
- 🔒 **Security** - No network exposure
- 🎯 **Simplicity** - Standard streams, no server setup
- 🔄 **Reliability** - Direct process communication

4. MCPHost Over Direct Ollama

- 🎓 **Learning Curve** - Easier to understand and debug
- 🛠️ **Development Experience** - Better tooling and error messages
- 🔧 **Flexibility** - Multiple AI providers, not just Ollama
- 🚀 **Production Ready** - More stable than experimental features

📋 Prerequisites & Setup

Required Software

- **Node.js** (v18+) - JavaScript runtime for our server
- **Git Bash** - Unix tools on Windows for `ls` command
- **Ollama** - Local AI model runtime
- **MCPHost** - MCP protocol host application
- **Go** - Required for installing MCPHost

Installation Steps

1. **Install Go and MCPHost**

```
# Install Go from https://golang.org
# Then install MCPHost
go install github.com/mark3labs/mcphost@latest
```

2. **Install and Setup Ollama**

```
# Install from https://ollama.ai
# Pull the model we'll use
ollama pull llama3.2:1b
```

3. **Project Setup**

```
git clone <your-repo>
cd simpl-mcp
npm install
npm run build
```

🚀 Running the Project

Automated Startup (Recommended)

```
.\start-mcphost.ps1
```

This script:

- ✅ Checks all prerequisites
- 🔧 Builds the project if needed
- 🚀 Starts MCPHost with proper configuration
- 📊 Shows helpful status information

Manual Startup (Educational)

```
# Terminal 1: Start Ollama service
ollama serve

# Terminal 2: Build and start our MCP server
npm run build

# Terminal 3: Start MCPHost (connects to our server automatically)
mcphost --config .mcphost.yml
```

🛠️ Configuration Explained

MCPHost Configuration (`.mcphost.yml`)

```yaml
# AI Model Configuration
model: "ollama:llama3.2:1b"              # Which AI model to use

# MCP Server Configuration
mcpServers:
  simple-directory-server:                # Our server name
    type: "local"                         # Local process (not remote HTTP)
    command: ["node", "dist/main.js"]     # How to start our server
    environment:
      NODE_ENV: "production"              # Environment variables
      DEBUG: "${env://DEBUG:-false}"      # With default values

# AI Model Parameters (Optimized for Llama 3.2:1b)
max-tokens: 2048                          # Response length limit
temperature: 0.7                          # Creativity (0.0-1.0)
top-p: 0.95                              # Diversity control
top-k: 40                                # Token selection limit

# System Behavior
max-steps: 0                             # Unlimited reasoning steps
debug: false                             # Debug logging
stream: true                             # Real-time responses
```

Why These Settings?

- **Llama 3.2:1b** - Small, fast model perfect for learning
- **Local type** - Runs our Node.js server as subprocess
- **Environment variables** - Flexible configuration without hardcoding
- **Optimized parameters** - Balanced performance for 1b model

🧪 Testing and Learning

1. Test MCP Server Independently

```
# Run our server directly (educational)
node dist/main.js
# Should output: "Simple MCP Directory Server running on stdio"
# Server is now waiting for JSON-RPC messages on stdin
```

2. Test with MCPHost

```
# Start the full system
mcphost

# Check available tools
/tools
# Should show: list_directory

# Test natural language
"List the files in C:/Users"
```

3. Example Conversations

Basic usage:

```
You: What files are in the current directory?
AI: I'll check the current directory contents for you.
[Calls list_directory with path: "."]
AI: Here are the files in the current directory: [shows results]
```

Path handling:

```
You: Show me C:/Program Files
AI: I'll list the contents of C:/Program Files.
[Calls list_directory with path: "C:/Program Files"]
AI: [Shows program directories and files]
```

🔍 Understanding the Code Flow

Complete Request Flow

1. **User Input** → MCPHost receives natural language
2. **AI Processing** → Llama 3.2:1b determines it needs a tool
3. **Tool Discovery** → MCPHost asks our server for available tools
4. **Tool Selection** → AI chooses `list_directory` tool
5. **Parameter Extraction** → AI determines path parameter from context
6. **MCP Call** → MCPHost sends JSON-RPC request via stdin
7. **Server Processing** → Our DirectoryService handles the request
8. **System Command** → Service executes Git Bash `ls` command
9. **Response Formatting** → Service formats output for MCP
10. **Return to Host** → JSON-RPC response sent via stdout
11. **AI Integration** → MCPHost gives result to AI model
12. **User Response** → AI formats final answer for user

Message Examples

Tool Discovery Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list"
}
```

Tool Discovery Response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [{
      "name": "list_directory",
      "description": "List contents of a directory using Git Bash ls command",
      "inputSchema": {
        "type": "object",
        "properties": {"path": {"type": "string"}},
        "required": ["path"]
      }
    }]
  }
}
```

Tool Execution Request:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "list_directory",
    "arguments": {"path": "C:/Users"}
  }
}
```

🎓 Educational Extensions

Beginner Projects

1. **Add File Reading Tool**

```
// Add to DirectoryService
async readFile(path: string): Promise<string> {
  // Implementation
}

// Add to MCP handlers in main.ts
```

2. **Add File Writing Tool**

```
async writeFile(path: string, content: string): Promise<string> {
  // Safe file writing implementation
}
```

3. **Add Directory Creation Tool**

```
async createDirectory(path: string): Promise<string> {
  // Directory creation with proper error handling
}
```

Intermediate Projects

1. **Multiple MCP Servers**
   - Create separate servers for different domains
   - File operations server
   - System information server
   - Network tools server
2. **Advanced Error Handling**
   - Permission checking before operations
   - Path validation and sanitization
   - User-friendly error messages
3. **Configuration Management**
   - Environment-specific configs
   - User preferences
   - Security policies

Advanced Projects

1. **Remote MCP Server**
   - Convert to HTTP transport
   - Add authentication

- Multi-tenant support

2. **Custom MCPHost Integration**
   - Build your own MCP host
   - Custom AI model integration
   - Specialized user interfaces

🐛 Troubleshooting Guide

Common Issues and Solutions

Issue: MCPHost can't find our server

```
# Check if built correctly
Test-Path "dist/main.js"  # Should return True
npm run build             # If false, rebuild
```

Issue: Git Bash not found

```
# Find Git Bash location
Get-Command bash -ErrorAction SilentlyContinue
# Update path in directory.service.ts if different
```

Issue: Ollama connection problems

```
# Check Ollama status
ollama list                    # Should show llama3.2:1b
ollama serve                   # Start if not running
ollama run llama3.2:1b        # Test model directly
```

Issue: Path handling errors

```
# Enable debug mode to see path processing
$env:DEBUG = "true"
mcphost --debug
```

Debug Mode

```
# Enable comprehensive debugging
$env:DEBUG = "true"
$env:NODE_ENV = "development"
mcphost --debug --config .mcphost.yml
```

📚 Learning Resources

MCP Protocol

- Official MCP Specification
- MCP SDK Documentation

MCPHost

- MCPHost Repository
- MCPHost Documentation

Ollama

- Ollama Documentation
- Llama Model Information

NestJS

- NestJS Documentation
- NestJS Fundamentals

🎯 Key Learning Outcomes

After working with this project, you should understand:

Technical Concepts

- ✅ **MCP Protocol** - How AI systems communicate with external tools
- ✅ **Stdio Transport** - Inter-process communication mechanisms
- ✅ **JSON-RPC** - Remote procedure call protocol
- ✅ **AI Tool Integration** - Practical AI application architecture

Practical Skills

- ✅ **Building MCP Servers** - Creating tools for AI systems
- ✅ **Configuration Management** - YAML configs and environment variables
- ✅ **Error Handling** - Robust system integration
- ✅ **System Integration** - Connecting different technologies

Architecture Patterns

- ✅ **Service-Oriented Design** - Clean separation of concerns
- ✅ **Protocol Implementation** - Standard communication patterns
- ✅ **Dependency Injection** - Modern application structure
- ✅ **Cross-Platform Development** - Consistent behavior across systems

🚀 Next Steps

Extend This Project

1. **Add More Tools** - File operations, system info, network tools
2. **Improve Error Handling** - Better user feedback and recovery
3. **Add Security** - Permission checking and input validation
4. **Performance Optimization** - Caching and async improvements

Explore Related Technologies

1. **Other AI Models** - Try different Ollama models or OpenAI
2. **Alternative Transports** - HTTP, WebSocket, or custom protocols
3. **Production Deployment** - Docker containers and orchestration
4. **Monitoring and Logging** - Observability and debugging tools

Build Your Own MCP Ecosystem

1. **Specialized Servers** - Domain-specific tool collections
2. **Custom Hosts** - Tailored AI interaction interfaces
3. **Integration Platforms** - Connect multiple AI systems
4. **Enterprise Solutions** - Production-ready MCP deployments

📁 Project Structure

```
src/
├── app.module.ts              # Main NestJS application module
├── main.ts                    # MCP server entry point with stdio transport
├── main_new.ts               # Alternative entry point (identical to main.ts)
└── directory/
    ├── directory.controller.ts # HTTP REST controller for testing
    ├── directory.module.ts    # NestJS module configuration
    └── directory.service.ts   # Core business logic for directory operations

dist/                          # Compiled TypeScript output
├── main.js                    # Built MCP server (executed by MCPHost)
└── ...                        # Other compiled files

Configuration Files:
├── .mcphost.yml               # MCPHost configuration (primary)
├── ollama-mcp-config.json     # Legacy Ollama config (reference)
├── package.json               # Node.js dependencies and scripts
├── tsconfig.json              # TypeScript configuration
└── nest-cli.json              # NestJS CLI configuration

Scripts:
├── start-mcphost.ps1          # Complete setup and startup script
├── restart.ps1                # Quick restart (PowerShell)
└── restart.sh                 # Quick restart (Bash)

Documentation:
└── README.md                  # This comprehensive guide
```

📄 License

MIT License - Feel free to use this project for learning and teaching!

🤝 Contributing

This is an educational project! Contributions welcome:

- 📚 Improve documentation and explanations
- 🐛 Fix bugs and add error handling
- ✨ Add new educational examples
- 🧪 Create additional learning exercises

Happy Learning! 🎓🚀